

PHYS 449- HMWK 7

Chat GPT was
used to help with
the assignment

$$1-a) \mathcal{L}(u, \lambda) = u^T S u - \lambda(u^T u - 1)$$

$$\frac{\partial}{\partial u} \mathcal{L}(u, \lambda) = \frac{\partial}{\partial u} (u^T S u) - \frac{\partial}{\partial u} (\lambda (u^T u - 1))$$

$$= 2 S u - 2 \lambda u = 0$$

$$S u = \lambda u \rightarrow u^T S u = \lambda$$

$\therefore u$ must be an eigenvector of S . The variance will be maximum when we set u equal to the eigenvector having the largest eigenvalue λ_1 .

b) The principal axis are the directions that maximize the variance of the projected data. These are the eigenvectors of S associated with the largest eigenvalues. Geometrically, the covariance ellipsoid aligns with these eigenvectors, and the variance along each such axis equals its eigenvalue, so the axes of greatest spread coincide with the eigenvectors.

PHYS 449 - Homework 7, Question 2

Student Name: Allen Mathew

Student ID: 20964892

Due Date: Tuesday, November 18, 3:00pm

ChatGPT was used to help generate the code

2. Apply both PCA and t-SNE for dimensional reduction on the MNIST dataset, reproducing the results shown in lecture.

```
In [1]: #imports, RNG, plotting helpers
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from time import time

# sklearn
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

# Optional: TorchVision for MNIST
try:
    from torchvision import datasets, transforms
    TORCHVISION_AVAILABLE = True
except Exception:
    TORCHVISION_AVAILABLE = False

rng = np.random.default_rng(42)

def stratified_subset(X, y, N, rng=rng):
    """Return a stratified subset of size N (balanced across labels as much as poss
    X = np.asarray(X); y = np.asarray(y)
    classes = np.unique(y)
    per = max(1, N // len(classes))
    idxs = []
    for c in classes:
        cand = np.where(y == c)[0]
        pick = rng.choice(cand, size=min(per, len(cand)), replace=False)
        idxs.append(pick)
    idxs = np.concatenate(idxs)
    # If we need a bit more to reach exactly N, top up randomly
    if len(idxs) < N:
        remaining = np.setdiff1d(np.arange(len(y)), idxs, assume_unique=False)
        extra = rng.choice(remaining, size=min(N - len(idxs), len(remaining)), repl
        idxs = np.concatenate([idxs, extra])
    # If overshoot, trim
    if len(idxs) > N:
        idxs = rng.choice(idxs, size=N, replace=False)
```

```

    return X[idxs], y[idxs]

def scatter_2d(Z2, y, title):
    plt.figure(figsize=(7, 6))
    sc = plt.scatter(Z2[:,0], Z2[:,1], c=y.astype(int), s=6, alpha=0.8)
    plt.title(title)
    plt.xlabel("dim 1"); plt.ylabel("dim 2")
    plt.tight_layout()
    plt.show()

def describe_labels(y):
    cnt = Counter(y)
    return dict(sorted(cnt.items(), key=lambda kv: int(kv[0])))

```

(a) First, use PCA to reduce some 784-dimensional MNIST digits to 2 dimensions and make a scatter plot where each point's color corresponds to the digit label. Start with a few thousand digits (e.g. $N = 2000$), and increase N as long as the computational time is reasonable.

```

In [2]: # Load MNIST as (X: [N,784], y: [N]) in numpy WITHOUT NumPy 2.0 warning
def load_mnist_numpy(limit=None, use_torchvision=True, normalize=True):
    """
    Returns X in float32 (0..1 if normalize else 0..255) and y as strings of digits
    If limit is not None, returns only the first 'limit' samples (unshuffled).
    """
    X, y = None, None

    if use_torchvision and TORCHVISION_AVAILABLE:
        from torchvision import datasets, transforms
        tfm = transforms.Compose([transforms.ToTensor()]) # -> torch.FloatTensor i
        ds_train = datasets.MNIST(root="./data", train=True, download=True, transfo

        # Use torch.stack(...).numpy() instead of np.array(...) to avoid NumPy 2.0
        imgs = [ds_train[i][0].squeeze(0) for i in range(len(ds_train))] # list of
        X_t = __import__("torch").stack(imgs, dim=0) # [N,28,28] torch tensor
        X = X_t.numpy() # shared-memory numpy view, no deprecation warning
        y = np.array([str(int(ds_train[i][1])) for i in range(len(ds_train))], dtype

    else:
        # Fallback via sklearn OpenML (requires internet)
        from sklearn.datasets import fetch_openml
        mnist = fetch_openml("mnist_784", version=1, as_frame=False)
        X = mnist["data"].reshape(-1, 28, 28).astype(np.float32) # [N,28,28]
        y = mnist["target"].astype(object)

        # If normalize=True for OpenML path, scale later along with torchvision pat

    if limit is not None:
        X = X[:limit]
        y = y[:limit]

    # Flatten to [N,784] and scale
    X = X.reshape(len(X), -1).astype(np.float32)
    if normalize:

```

```

        # torchvision path already in [0,1]; OpenML in [0,255]. This works for both
        if X.max() > 1.5: # crude check to avoid double-normalizing
            X /= 255.0

    return X, y

# quick smoke check
X_full, y_full = load_mnist_numpy(limit=60000, use_torchvision=True, normalize=True)
print("Loaded:", X_full.shape, "label balance (first 10k):", dict(list(describe_lab

```

Loaded: (60000, 784) label balance (first 10k): {'0': 1001, '1': 1127, '2': 991, '3': 1032, '4': 980} ...

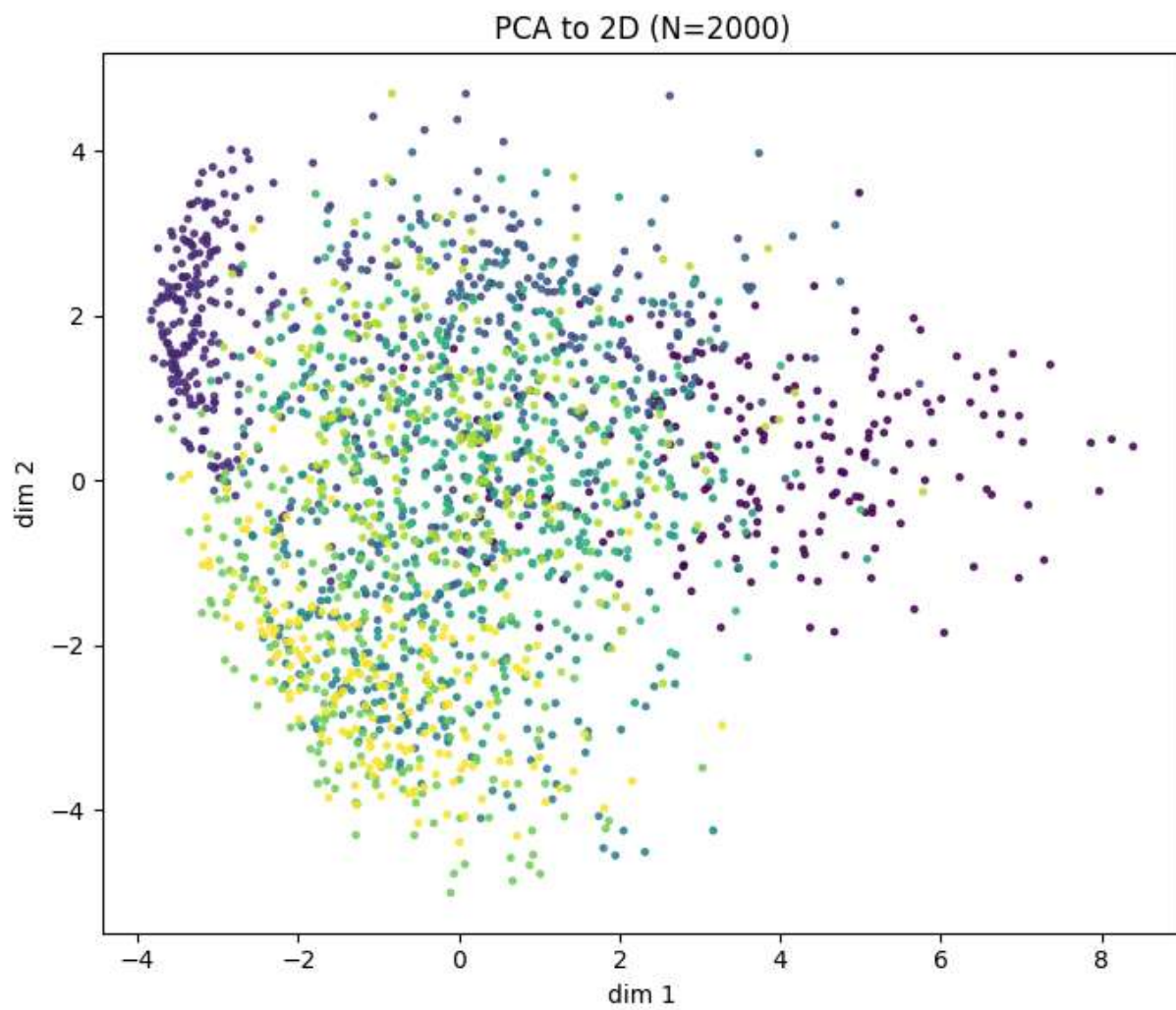
```

In [6]: # PCA experiments
def run_pca_at_sizes(X_full, y_full, sizes=(2000, 5000, 10000, 20000)):
    for N in sizes:
        XN, yN = stratified_subset(X_full, y_full, N)
        t0 = time()
        pca = PCA(n_components=2, svd_solver="randomized", random_state=42)
        Z2 = pca.fit_transform(XN)
        t1 = time()
        print(f"PCA N={N}: explained_var={pca.explained_variance_ratio_.sum():.4f},
              scatter_2d(Z2, yN, title=f"PCA to 2D (N={N})")

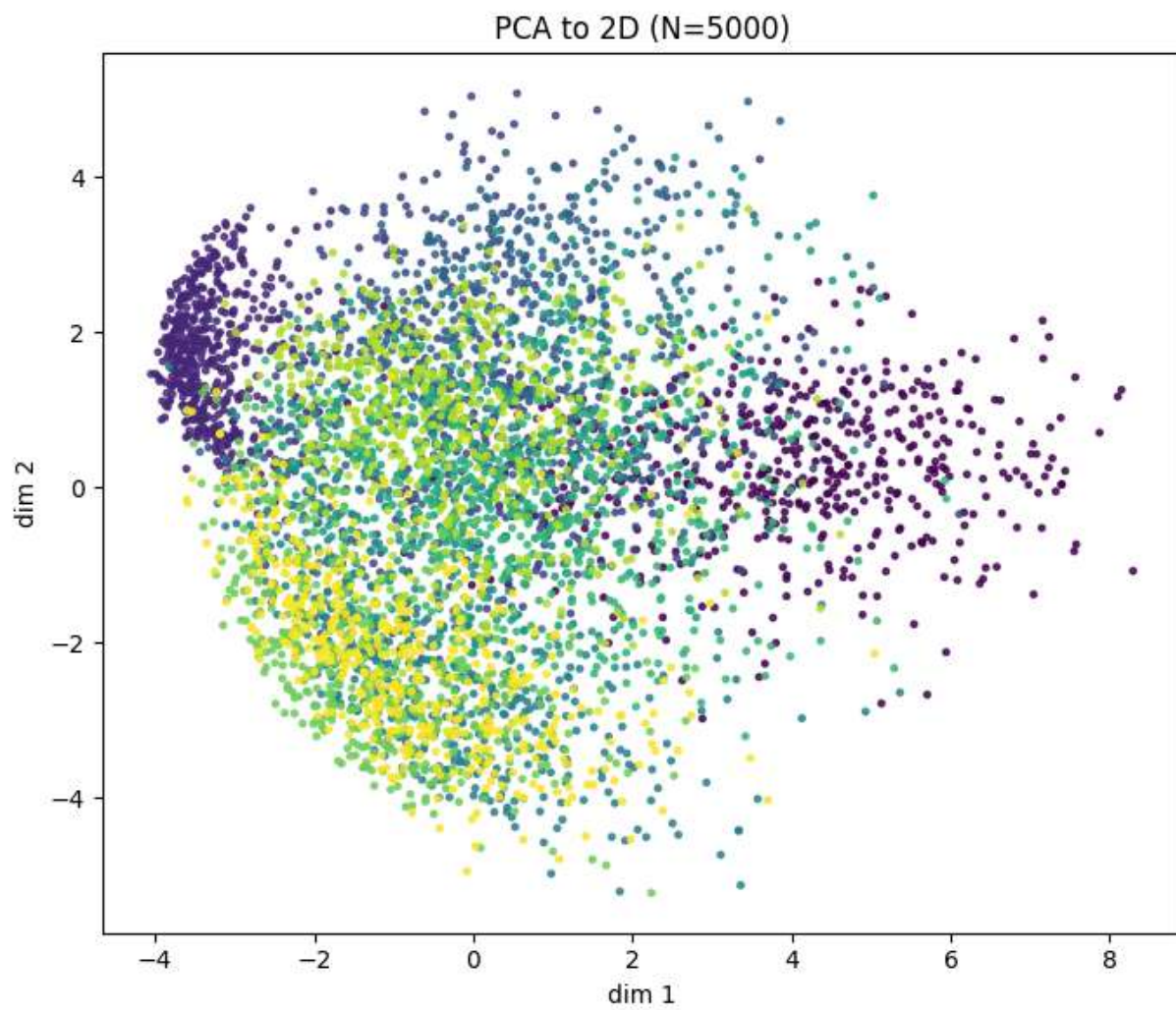
run_pca_at_sizes(X_full, y_full, sizes=(2000, 5000, 10000, 20000))

```

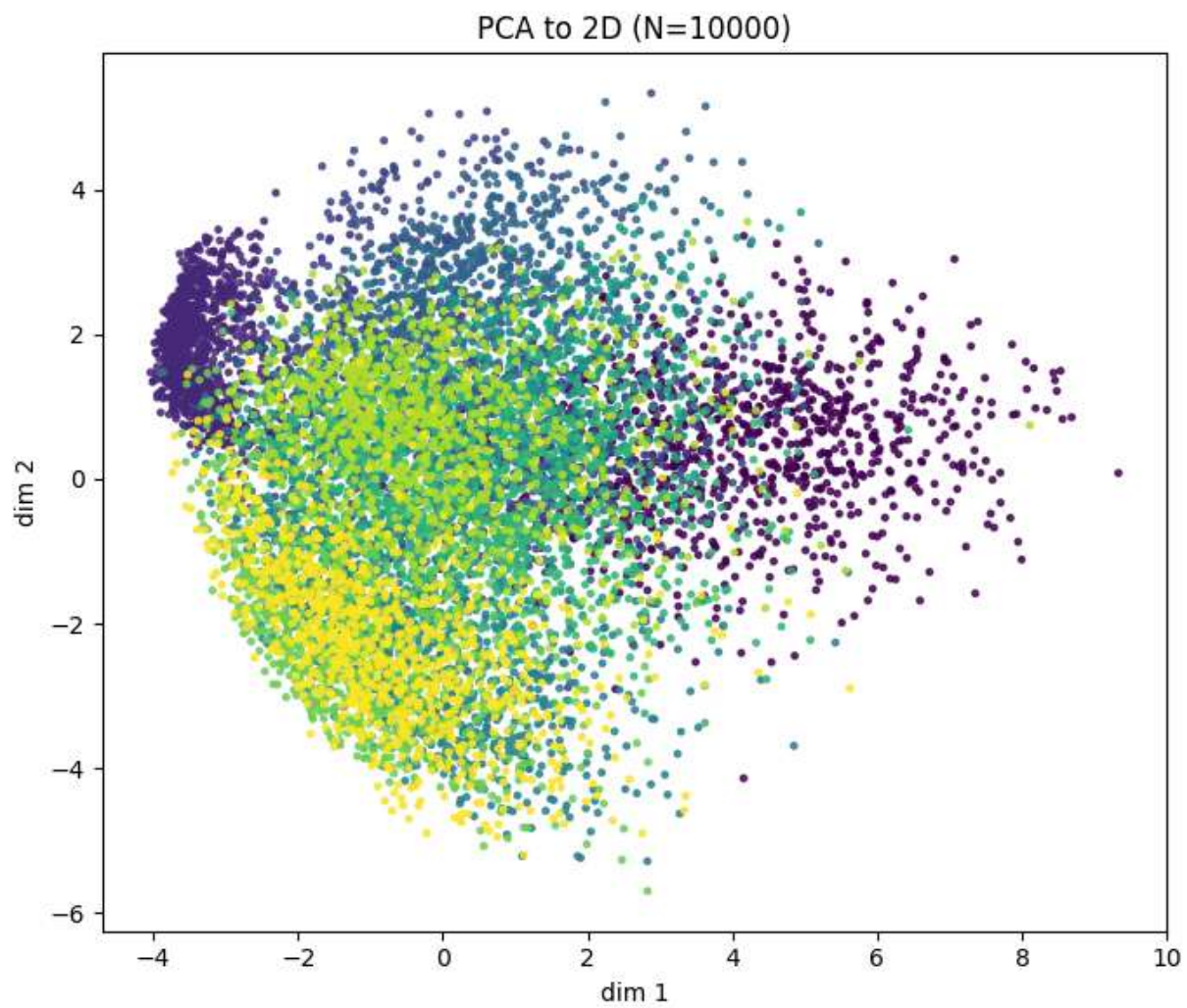
PCA N=2000: explained_var=0.1621, time=0.01s



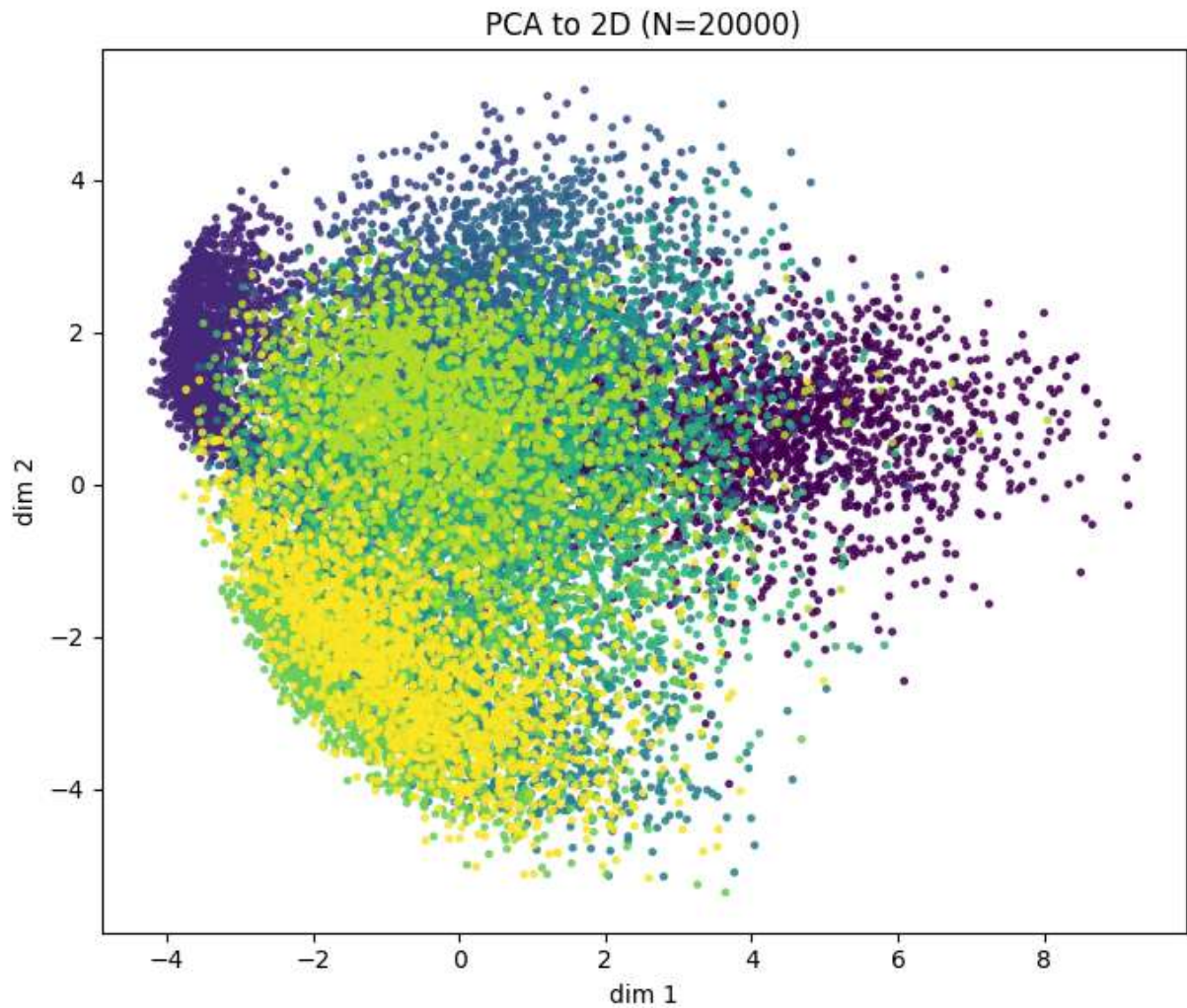
PCA N=5000: explained_var=0.1673, time=0.02s



PCA N=10000: explained_var=0.1660, time=0.07s



PCA N=20000: explained_var=0.1646, time=0.14s



(b) Similarly, use t-SNE to reduce the MNIST dataset to 2 dimensions and illustrate with scatter plots for a few different N .

```
In [7]: # t-SNE with version-robust args
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from time import time

def _make_tsne(perplexity, lr, iters, seed=42, metric="euclidean", verbose=1):
    base_kwargs = dict(
        n_components=2,
        perplexity=perplexity,
        learning_rate=lr,
        init="pca",
        random_state=seed,
        metric=metric,
        verbose=verbose,
        # don't pass n_jobs; not supported in many versions
    )
    # Try different iteration parameter names depending on sklearn version
    try:
        return TSNE(**base_kwargs, n_iter=iters)
    except TypeError:
        # sklearn <= 1.5
```

```

    try:
        return TSNE(**base_kwargs, max_iter=iters)    # if newer sklearn rename
    except TypeError:
        return TSNE(**base_kwargs)                  # fall back to default (u

def run_tsne_at_sizes(X_full, y_full, sizes=(2000, 5000, 10000, 20000), perplexity=
"""
Note: t-SNE is  $O(N^2)$ . Keep N moderate unless you use a faster implementation.
"""
for N in sizes:
    XN, yN = stratified_subset(X_full, y_full, N)
    # Speed-up: PCA to 50D before t-SNE
    XN_50 = PCA(n_components=50, svd_solver="randomized", random_state=42).fit_u

    tsne = _make_tsne(perplexity=perplexity, lr=lr, iters=iters, seed=42)
    t0 = time()
    Z2 = tsne.fit_transform(XN_50)
    t1 = time()
    try:
        iters_used = getattr(tsne, "n_iter_", None)
    except Exception:
        iters_used = None
    msg = f"t-SNE N={N}: time={t1-t0:.2f}s"
    if iters_used is not None:
        msg += f" (effective iters={iters_used})"
    print(msg)
    scatter_2d(Z2, yN, title=f"t-SNE to 2D (N={N}, perplexity={perplexity})")

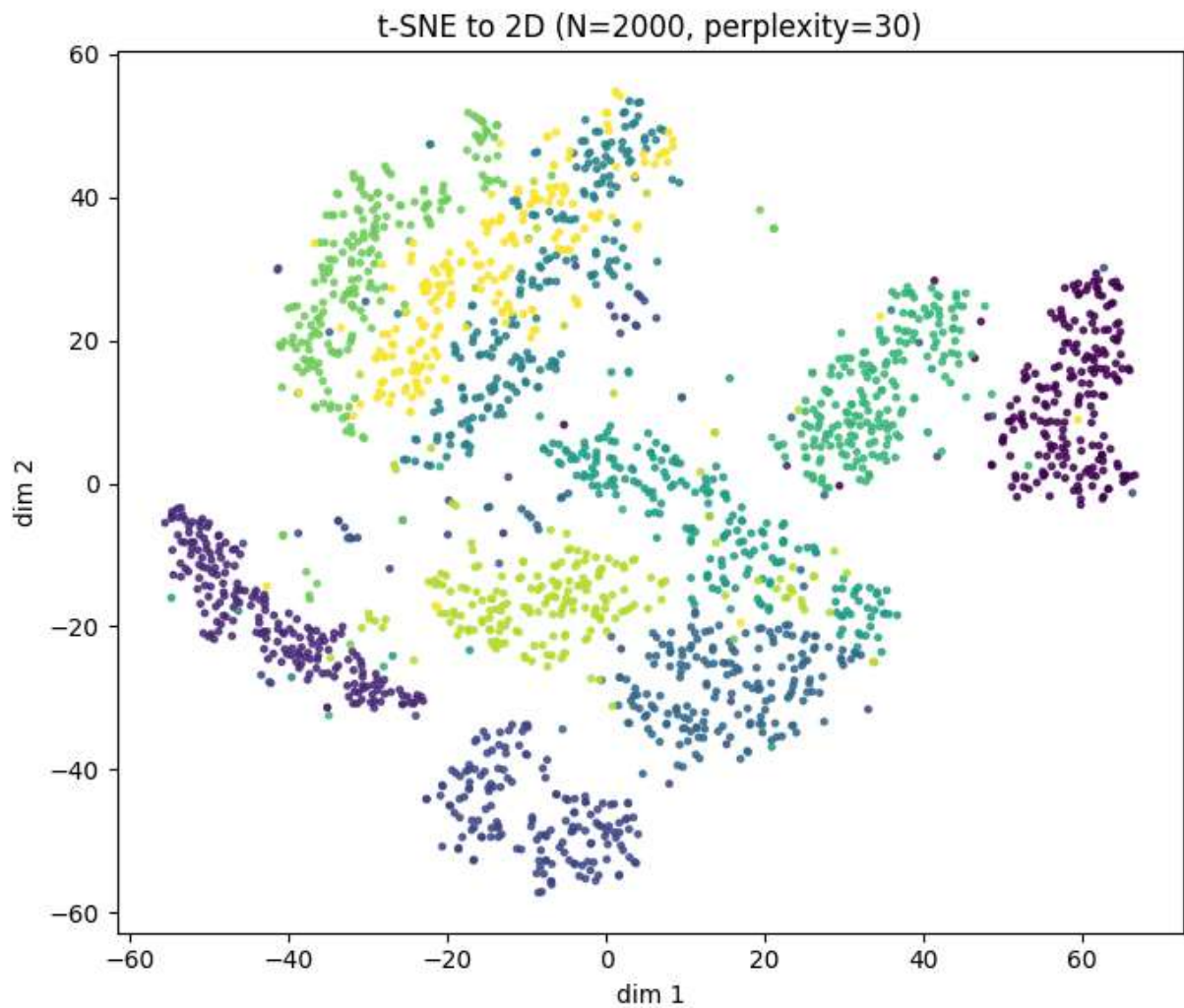
run_tsne_at_sizes(X_full, y_full, sizes=(2000, 5000, 10000, 20000), perplexity=30,

```

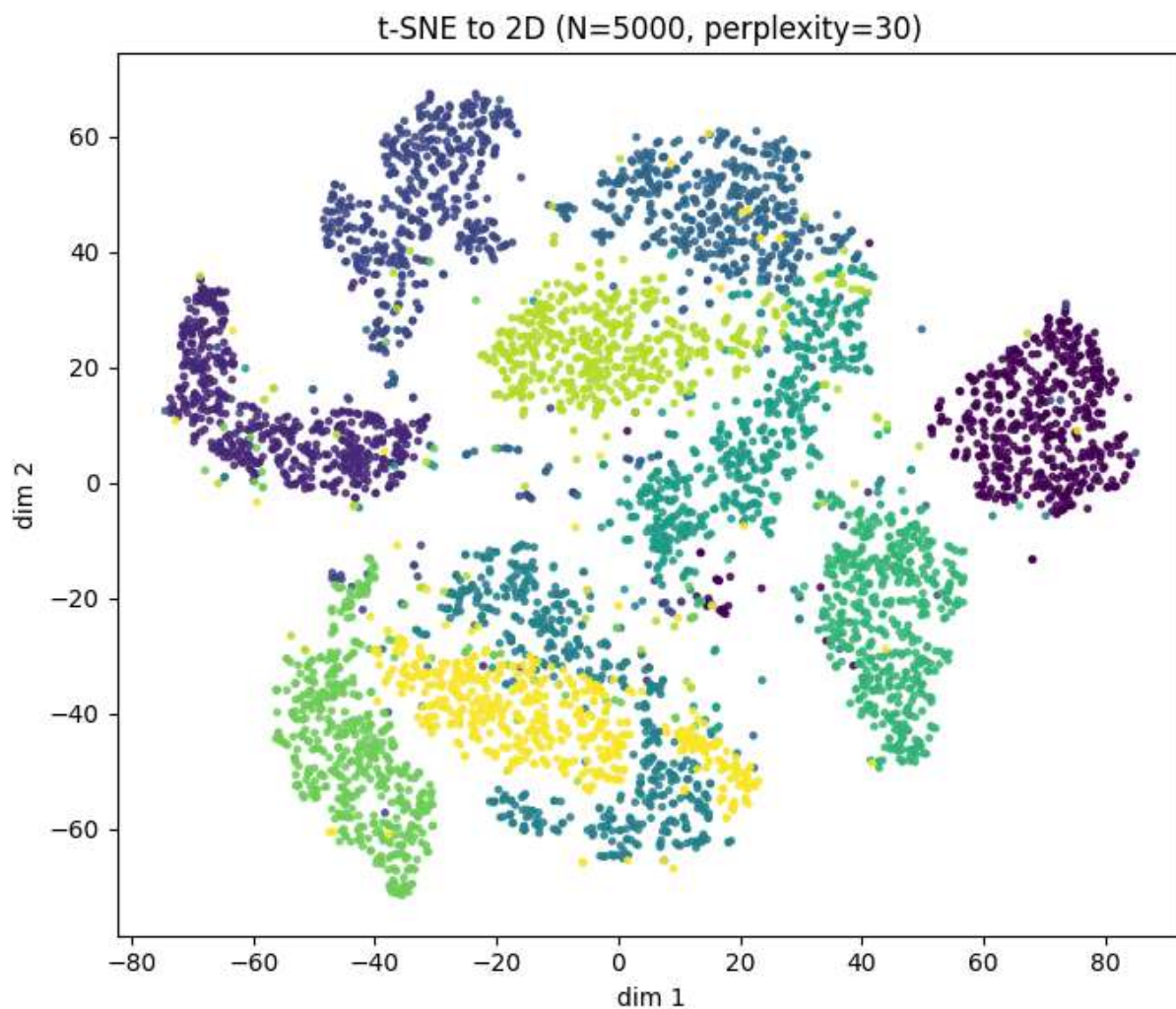
```

[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 2000 samples in 0.000s...
[t-SNE] Computed neighbors for 2000 samples in 0.030s...
[t-SNE] Computed conditional probabilities for sample 1000 / 2000
[t-SNE] Computed conditional probabilities for sample 2000 / 2000
[t-SNE] Mean sigma: 2.311693
[t-SNE] KL divergence after 250 iterations with early exaggeration: 75.389534
[t-SNE] KL divergence after 1000 iterations: 1.208266
t-SNE N=2000: time=3.57s (effective iters=999)

```

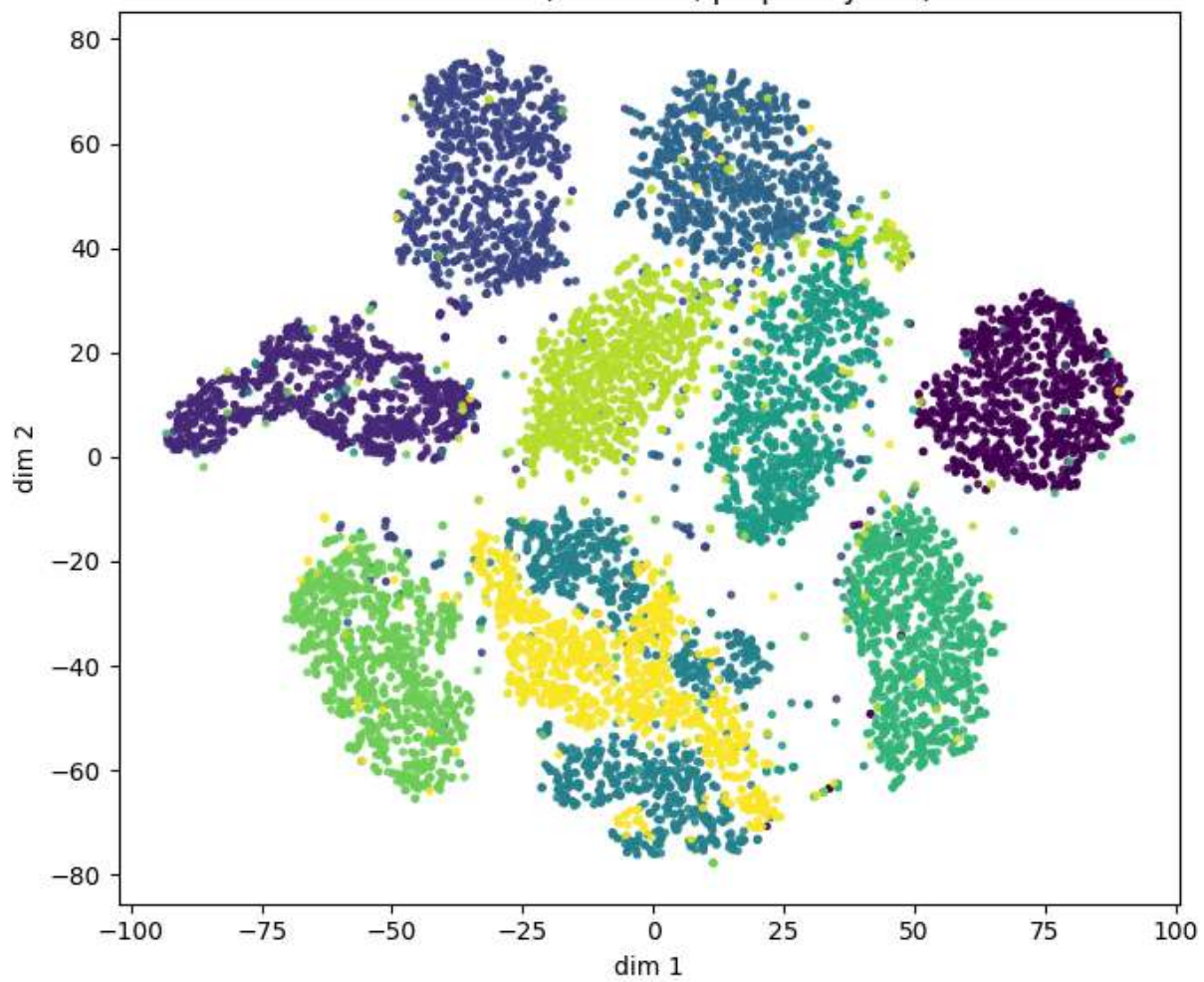


```
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 5000 samples in 0.001s...
[t-SNE] Computed neighbors for 5000 samples in 0.114s...
[t-SNE] Computed conditional probabilities for sample 1000 / 5000
[t-SNE] Computed conditional probabilities for sample 2000 / 5000
[t-SNE] Computed conditional probabilities for sample 3000 / 5000
[t-SNE] Computed conditional probabilities for sample 4000 / 5000
[t-SNE] Computed conditional probabilities for sample 5000 / 5000
[t-SNE] Mean sigma: 1.968403
[t-SNE] KL divergence after 250 iterations with early exaggeration: 81.972404
[t-SNE] KL divergence after 1000 iterations: 1.498033
t-SNE N=5000: time=10.27s (effective iters=999)
```

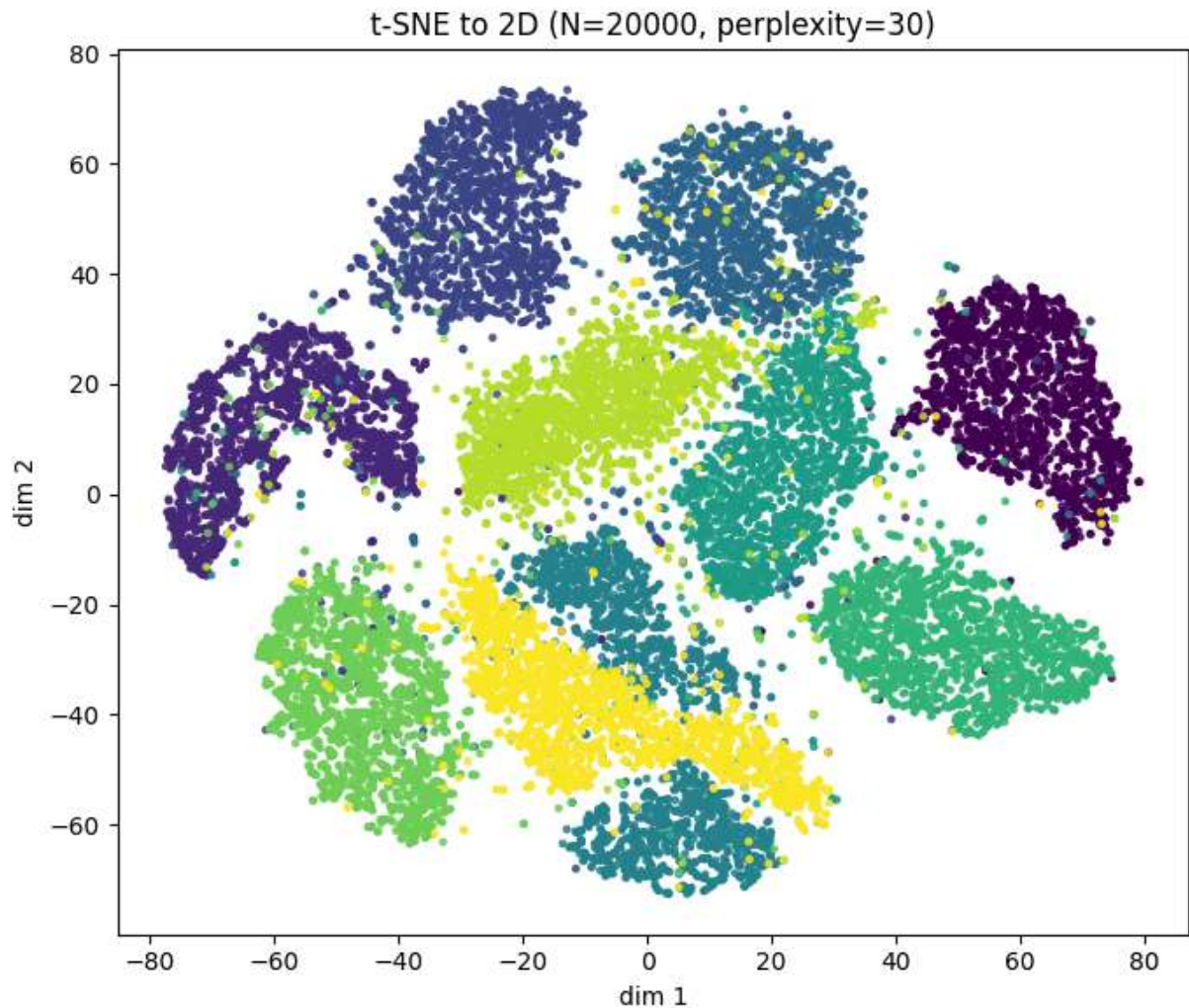


```
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 10000 samples in 0.000s...
[t-SNE] Computed neighbors for 10000 samples in 0.157s...
[t-SNE] Computed conditional probabilities for sample 1000 / 10000
[t-SNE] Computed conditional probabilities for sample 2000 / 10000
[t-SNE] Computed conditional probabilities for sample 3000 / 10000
[t-SNE] Computed conditional probabilities for sample 4000 / 10000
[t-SNE] Computed conditional probabilities for sample 5000 / 10000
[t-SNE] Computed conditional probabilities for sample 6000 / 10000
[t-SNE] Computed conditional probabilities for sample 7000 / 10000
[t-SNE] Computed conditional probabilities for sample 8000 / 10000
[t-SNE] Computed conditional probabilities for sample 9000 / 10000
[t-SNE] Computed conditional probabilities for sample 10000 / 10000
[t-SNE] Mean sigma: 1.788028
[t-SNE] KL divergence after 250 iterations with early exaggeration: 86.369095
[t-SNE] KL divergence after 1000 iterations: 1.769090
t-SNE N=10000: time=25.86s (effective iters=999)
```

t-SNE to 2D (N=10000, perplexity=30)



```
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 20000 samples in 0.003s...
[t-SNE] Computed neighbors for 20000 samples in 0.657s...
[t-SNE] Computed conditional probabilities for sample 1000 / 20000
[t-SNE] Computed conditional probabilities for sample 2000 / 20000
[t-SNE] Computed conditional probabilities for sample 3000 / 20000
[t-SNE] Computed conditional probabilities for sample 4000 / 20000
[t-SNE] Computed conditional probabilities for sample 5000 / 20000
[t-SNE] Computed conditional probabilities for sample 6000 / 20000
[t-SNE] Computed conditional probabilities for sample 7000 / 20000
[t-SNE] Computed conditional probabilities for sample 8000 / 20000
[t-SNE] Computed conditional probabilities for sample 9000 / 20000
[t-SNE] Computed conditional probabilities for sample 10000 / 20000
[t-SNE] Computed conditional probabilities for sample 11000 / 20000
[t-SNE] Computed conditional probabilities for sample 12000 / 20000
[t-SNE] Computed conditional probabilities for sample 13000 / 20000
[t-SNE] Computed conditional probabilities for sample 14000 / 20000
[t-SNE] Computed conditional probabilities for sample 15000 / 20000
[t-SNE] Computed conditional probabilities for sample 16000 / 20000
[t-SNE] Computed conditional probabilities for sample 17000 / 20000
[t-SNE] Computed conditional probabilities for sample 18000 / 20000
[t-SNE] Computed conditional probabilities for sample 19000 / 20000
[t-SNE] Computed conditional probabilities for sample 20000 / 20000
[t-SNE] Mean sigma: 1.591082
[t-SNE] KL divergence after 250 iterations with early exaggeration: 90.462883
[t-SNE] KL divergence after 1000 iterations: 2.163366
t-SNE N=20000: time=55.01s (effective iters=999)
```



(c) Discuss your results. Which digits are well separated by each method? What N do you need to see well-defined clusters in each case? What are the advantages and disadvantages of each method?

For PCA, we can see that clusters overlap heavily. This is because PCA spreads digits along global factors like stroke thickness and direction. t-SNE has well separated islands corresponding to each digit. Thus, t-SNE separates digits better than PCA. For PCA, we needed $N \approx 20000$ to really see well-defined clusters since it's really hard to see changes through moderate increases of N . However, t-SNE needed $N \approx 5000$ to see well-defined clusters. In terms of pros and cons, PCA is ridiculously fast and can show general stroke patterns for every digit but cannot separate digits well. t-SNE is slower, but can clearly separate digits.